

Class Activity: Needleman-Wunsch Algorithm for Sequence Alignment

Activity Title: Global Sequence Alignment using the Needleman-Wunsch Algorithm

Learning Objectives:

By the end of this activity, students should be able to:

- ✓ Understand the **Needleman-Wunsch algorithm** for global sequence alignment.
 - ✓ Implement the algorithm using **Biopython**.
 - ✓ Perform alignment of two DNA or protein sequences.
 - ✓ Solve exercises related to **sequence alignment scoring and traceback**.
-

Activity Overview

This class activity consists of **three parts**:

1. **Concept Explanation** – Understanding the Needleman-Wunsch algorithm.
 2. **Hands-on Coding Exercise** – Implementing and running alignments using Biopython.
 3. **Group Challenge & Exercises** – Solving alignment problems in teams and analyzing results.
-

Part 1: Concept Explanation (15 minutes)

1. Introduction to Sequence Alignment

- **Sequence alignment** compares two sequences to find similarities.
- **Global alignment** (Needleman-Wunsch) aligns sequences from start to end.
- **Local alignment** (Smith-Waterman) aligns only similar subsequences.

2. The Needleman-Wunsch Algorithm

The **Needleman-Wunsch** algorithm follows three main steps:

1. **Initialization** – Create a scoring matrix with gap penalties.
2. **Filling the Matrix** – Compute scores based on matches, mismatches, and gaps.
3. **Traceback** – Recover the optimal alignment.

Scoring System:

- **Match:** +1
- **Mismatch:** -1
- **Gap penalty:** -2

Example Sequences:

Let's align "GATTACA" and "GCATGCU".

Part 2: Hands-on Coding Exercise (20 minutes)

1. Implementing Needleman-Wunsch in Biopython

```
from Bio import pairwise2
from Bio.pairwise2 import format_alignment

# Define sequences
seq1 = "GATTACA"
seq2 = "GCATGCU"

# Perform global alignment
alignments = pairwise2.align.globalxx(seq1, seq2)

# Print results
for alignment in alignments:
    print(format_alignment(*alignment))
```

 **Expected Output:**

Part 3: Group Challenge & Exercises (25 minutes)

Group Challenge (10 minutes)

1. Form small groups (2-3 students).
2. Each group is given **two sequences** to align.
3. They must **calculate the alignment matrix manually** for a small sequence pair.
4. **Verify results** by running their calculations in Biopython.

Example Sequences for Groups

- Group 1: "ATGC" vs. "AGC"
- Group 2: "GACT" vs. "GAT"
- Group 3: "TGCAG" vs. "TGAC"
- Group 4: "ACGTAC" vs. "ACTGAC"

Discussion: How do **gap penalties** affect the alignment?

Individual Exercises (15 minutes)

Exercise 1: Identify Optimal Alignment

For "AGCTA" and "GCT", compute the **optimal alignment score** manually using **match = 2**, **mismatch = -1**, **gap = -2**.

Exercise 2: Custom Alignment Function

Write a Python function that:

- Takes **two sequences** as input.
- Uses `pairwise2.align.globalms()` with **custom scoring**.
- Returns the **best alignment only**.

Exercise 3: Effect of Gap Penalty

Run alignments with different **gap penalties** (-1, -2, -3, -5) and analyze how the results change.

Expected Learning Outcomes

- ✓ Understand the **Needleman-Wunsch algorithm** conceptually and practically.
 - ✓ Implement global sequence alignment using **Biopython**.
 - ✓ Interpret **alignment scores and traceback results**.
 - ✓ Analyze the effects of **different scoring systems**.
-

Discussion Questions

1. Why does increasing the **gap penalty** lead to shorter alignments?
 2. What happens when sequences **do not align well**?
 3. How does this algorithm differ from **local alignment (Smith-Waterman)**?
-

Additional Resources

- **Biopython Documentation:** <https://biopython.org/wiki/Documentation>
- **NCBI Resources:** <https://www.ncbi.nlm.nih.gov/>